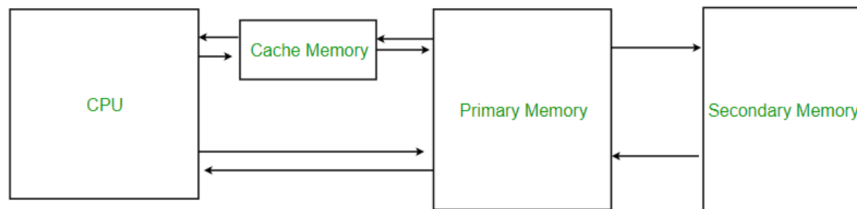


Tutorial 8

SC1006, CZ1106

Cache and Virtual Memory

Cache Memory is a special very high-speed memory. It is used to speed up and synchronizing with high-speed CPU. Cache memory is costlier than main memory or disk memory but economical than CPU registers. Cache memory is an extremely fast memory type that acts as a buffer between RAM and the CPU. It holds frequently requested data and instructions so that they are immediately available to the CPU when needed.



There are three different policies available for placement of a memory block in the cache: **direct-mapped**, **fully associative**, and **set-associative**.

In a **direct-mapped** cache structure, the cache is organized into multiple sets with a single cache line per set. Based on the address of the memory block, it can only occupy a single cache line.

When the processor needs to read or write a location in main memory, it first checks for a corresponding entry in the cache.

- If the processor finds that the memory location is in the cache, a **cache hit** has occurred and data is read from cache
- If the processor **does not** find the memory location in the cache, a **cache miss** has occurred. For a cache miss, the cache allocates a new entry and copies in data from main memory, then the request is fulfilled from the contents of the cache.

The performance of cache memory is frequently measured in terms of a quantity called **Hit ratio**.

$$\text{Hit ratio} = \text{hit} / (\text{hit} + \text{miss}) = \text{no. of hits} / \text{total accesses}$$

We can improve Cache performance using higher cache block size, higher associativity, reduce miss rate, reduce miss penalty, and reduce the time to hit in the cache.

1. Given a processor system with the following characteristics

- Processor has a direct-mapped cache with 32 cache blocks and a cache size of 512 bytes.
- Cache Memory Access time = 5ns.
- Cache Hit rate = 0.9
- 64Kbyte DRAM used as the main memory.
- DRAM Memory access time = 200ns

a. In doing cache mapping analysis, how many **blocks** would the main memory be partitioned to?

$$\text{Cache block size} = 512/32 = 16 \text{ Bytes}$$

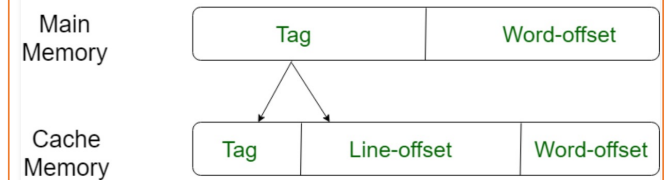
$$\text{Number of main memory blocks} = 64\text{KByte}/16\text{Byte} = (2^{16})/(2^4) = 2^{12} = 4096$$

1. Given a processor system with the following characteristics

- Processor has a direct-mapped cache with 32 cache blocks and a cache size of 512 bytes.
- Cache Memory Access time = 5ns.
- Cache Hit rate = 0.9
- 64Kbyte DRAM used as the main memory.
- DRAM Memory access time = 200ns

When a cache block is copied from memory into the cache, a cache entry is created. The cache entry will include the copied data as well as the requested memory location (called a tag).

For purposes of cache access, each main memory address can be viewed as consisting of three fields. The least significant w bits identify a unique word or byte within a block of main memory. In most contemporary machines, the address is at the byte level. The remaining s bits specify one of the 2^s blocks of main memory. The cache logic interprets these s bits as a tag of $s-r$ bits (most significant portion) and a line field of r bits. This latter field identifies one of the $m=2^r$ lines of the cache.



b. What is the format of a memory address as seen by the cache (i.e. determine the sizes of the tag, block and offset fields)?

Cache Block Size = 16 $\Rightarrow$ Offset Field = 4 bits

$$2^4=16$$

Number of blocks in the cache = 32 $\Rightarrow$ Block Field = 5 bits

$$2^5=32$$

Main Memory Size = 64KByte $\Rightarrow$ 16 address bits

$$2^{16}=65,536$$

Tag Field bits = Main Memory Address bits – Offset – Block = 7 (16-4-5=7)

$\Rightarrow$ TAG:BLOCK:OFFSET = 7:5:4

c. CPU needs to read a byte from main memory address 0x0DB63.

- i. Which cache block would CPU look at to search for the required data?
- ii. How many main memory blocks could potentially be mapped to the same cache block as that of 0x0DB63?
- iii. How does the CPU know if the cache block identified in (i) above contains the data that it needs?
- iv. What is the purpose of the 'offset' field in the cache mapping?

i. $0x0DB63 = 1101\ 1011\ 0110\ 0011$

$\Rightarrow$ Block 10110b = Block 22

TAG:BLOCK:OFFSET = 7:5:4

ii. Number of MM blocks = 4096

Number of cache blocks = 32

$\Rightarrow$ each cache block is a potential destination to $4096/32 = 128$ MM blocks.

iii. By looking at the TAG value entry of the corresponding cache block
For 0xDB63, the TAG value for the cache block should be equal to
1101 101b

iv. Offset refers to the offset of the byte of interest from the cache block boundary. For 0xDB63, the byte is the byte 3 of block 22. First byte of the block is byte 0.

Processor has a direct-mapped cache with 32 cache blocks

Number of main memory blocks = $64\text{KByte}/16\text{Byte} = (2^{16})/(2^4) = 2^{12} = 4096$

d. What is the effective access time of the memory in this system?

The performance of cache memory is frequently measured in terms of a quantity called **Hit ratio**.

$$\text{Hit ratio} = \text{hit} / (\text{hit} + \text{miss}) = \text{no. of hits} / \text{total accesses}$$

We can improve Cache performance using higher cache block size, higher associativity, reduce miss rate, reduce miss penalty, and reduce the time to hit in the cache.

Assume cache memory and main memory access do not overlap.

$$\text{EAT} = H * \text{Cache}_{\text{Access}} + (1-H) * (\text{Cache}_{\text{Access}} + \text{Mem}_{\text{Access}}) = 0.9 * 5\text{ns} + (1-0.9) * (5\text{ns} + 200\text{ns}) = 25\text{ns}.$$

Cache Hit rate = 0.9

Cache Memory Access time = 5ns.

DRAM Memory access time = 200ns

Virtual memory is a feature of an operating system that enables a computer to be able to compensate shortages of physical memory by transferring pages of data from random access memory to disk storage.

This process is done temporarily and is designed to work as a combination of RAM and space on the hard disk.

This means that when RAM runs low, virtual memory can move data from it to a space called a paging file. This process allows for RAM to be freed up so that a computer can complete the task.

A translation lookaside buffer (TLB) is a memory cache that stores the recent translations of virtual memory to physical memory. It is used to reduce the time taken to access a user memory location. It can be called an address-translation cache. It is a part of the chip's memory-management unit (MMU). A TLB may reside between the CPU and the CPU cache, between CPU cache and the main memory or between the different levels of the multi-level cache. The majority of desktop, laptop, and server processors include one or more TLBs in the memory-management hardware, and it is nearly always present in any processor that utilizes paged or segmented virtual memory.

2. In a processor system with the following characteristics,

- 1 MByte Virtual memory space
- 64 Kbyte DRAM as main memory
- Paging scheme used for virtual memory management, Page Table as shown in Table 8.2
- Virtual Page size = 1 KByte
- TLB with 4 entries

Table 8.2 – Page Table

Virtual Page Number	Valid Bit	Page Frame Number
0	1	1
1	1	2
2	0	-
3	1	16
4	1	9

a. How many bits are required for each virtual address?

1Mbyte = 2^{20} byte $\Rightarrow$ 20 bits

2. In a processor system with the following characteristics,

- 1 MByte Virtual memory space
- 64 Kbyte DRAM as main memory
- Paging scheme used for virtual memory management, Page Table as shown in Table 8.2
- Virtual Page size = 1 KByte
- TLB with 4 entries

Table 8.2 – Page Table

Virtual Page Number	Valid Bit	Page Frame Number
0	1	1
1	1	2
2	0	-
3	1	16
4	1	9

b. How many bits are required for each physical address?

$$2^6=64$$

$$64 \text{ KByte} = 2^6 * 2^{10} = 2^{16} \Rightarrow 16 \text{ bits}$$

2. In a processor system with the following characteristics,

- 1 MByte Virtual memory space
- 64 Kbyte DRAM as main memory
- Paging scheme used for virtual memory management, Page Table as shown in Table 8.2
- Virtual Page size = 1 KByte
- TLB with 4 entries

Table 8.2 – Page Table

Virtual Page Number	Valid Bit	Page Frame Number
0	1	1
1	1	2
2	0	-
3	1	16
4	1	9

c. What is the maximum number of entries in the page table in Table 8.2?

$$\text{Number of virtual pages} = 2^{20} / 2^{10} = 2^{10} = 1024$$

$$\Rightarrow \text{Max number of entries} = 1024$$

2. In a processor system with the following characteristics,

- 1 MByte Virtual memory space
- 64 Kbyte DRAM as main memory
- Paging scheme used for virtual memory management, Page Table as shown in Table 8.2
- Virtual Page size = 1 KByte
- TLB with 4 entries

Table 8.2 – Page Table

Virtual Page Number	Valid Bit	Page Frame Number
0	1	1
1	1	2
2	0	-
3	1	16
4	1	9

How many bits are required for each physical address?

$$2^6=64$$

$$64 \text{ KByte} = 2^6 * 2^{10} = 2^{16} \Rightarrow 16 \text{ bits}$$

d. What is the maximum number of valid entries in the page table in Table 8.2?

$$\text{Number of Physical Frames} = 2^{16} / 2^{10} = 64$$

$$\Rightarrow \text{Max number of valid entries} = 64$$

e. With reference to Table 8.2, answer the following. Indicate when a page fault occurs.

- (i) The compiler mapped the UART routine to virtual address 0x005F0 – 0x006FF, where in the DRAM would you be able to find the UART routine?

page_number:offset

0x005F0 → virtual page number 1 → mapped to page frame number 2

0x006FF → virtual page number 1 → mapped to page frame number 2

Virtual Address (20 bits)	Physical Address (16 bits)
0000 0000 0101 1111 0000	0000 1001 1111 0000
0000 0000 0110 1111 1111	0000 1010 1111 1111

Physical Address = 0x09F0 – 0x0AFF

Table 8.2 – Page Table

Virtual Page Number	Valid Bit	Page Frame Number
0	1	1
1	1	2
2	0	-
3	1	16
4	1	9

Virtual Page size = 1 KByte i.e. requires 10 bits

(ii) The compiler mapped the I2C routine to virtual address 0x009C0 - 0x009DF, where in the DRAM would you be able to find the I2C routine?

1001 1100 0000 – 1001 1101 1111

0x009C0 → virtual page number 2 → page fault occurs (valid bit =0).

0x009DF → virtual page number 2 → page fault occurs (valid bit =0).

Table 8.2 – Page Table

Virtual Page Number	Valid Bit	Page Frame Number
0	1	1
1	1	2
2	0	-
3	1	16
4	1	9

(iii) What happens when there is a page fault?

Page fault => the required data/code is not in the main memory

=> OS needs to retrieve the required information from the storage memory and update the paging table accordingly.

f. What memory are the Page Table and TLB resided?

Main Page Table => Main Memory

TLB => Internal Fast Memory close to CPU

A translation lookaside buffer (TLB) is a memory cache that stores the recent translations of virtual memory to physical memory. It is used to reduce the time taken to access a user memory location. It can be called an address-translation cache. It is a part of the chip's memory-management unit (MMU). A TLB may reside between the CPU and the CPU cache, between CPU cache and the main memory or between the different levels of the multi-level cache. The majority of desktop, laptop, and server processors include one or more TLBs in the memory-management hardware, and it is nearly always present in any processor that utilizes paged or segmented virtual memory.

g. What is the function and effect of a TLB?

- ⇒ Cache the frequently used Page table information
- ⇒ Leads to shorter access time for paging information => increase in system performance.

A translation lookaside buffer (TLB) is a memory cache that stores the recent translations of virtual memory to physical memory. It is used to reduce the time taken to access a user memory location. It can be called an address-translation cache. It is a part of the chip's memory-management unit (MMU). A TLB may reside between the CPU and the CPU cache, between CPU cache and the main memory or between the different levels of the multi-level cache. The majority of desktop, laptop, and server processors include one or more TLBs in the memory-management hardware, and it is nearly always present in any processor that utilizes paged or segmented virtual memory.

3. Consider a system with the following characteristics.

- Direct mapped cache of 32 cache blocks and cache block size of 32 bytes
- Cache uses Physical Address for address mapping
- Virtual Memory page size 2048 bytes 2^{11}
- Virtual Memory size is 1Mbyte. Physical Memory size is 64KByte
- Extracts of Page Table (valid entries)
 - Virtual Page 0 → Physical Frame 9
 - Virtual Page 1 → Physical Frame 3
 - Virtual Page 2 → Physical Frame 5
 - Virtual Page 3 → Physical Frame 2
 - Virtual Page 4 → Physical Frame 7
- The main program is 5KByte in size and starts at virtual address 0x01006

(i) Assuming that the compiler allocates the program sequentially in the virtual memory, what is the physical address of the start and end of the main program?

5 Kbytes=5,120 bytes (decimal)

0x01006=4,102 (decimal)

4,102+5,120=9,222 (decimal)

9,222-1=9,221 (decimal address

of the last byte) = 0x02405 or

0010 0100 0000 0101 (binary)

Virtual Address (Start) = 0x01006, end = 0x02405. (5KByte size)

0x1006 => 0001 0000 0000 0110 => Page 2 => Physical Frame 5 (101b)

Physical address = 0010 1000 0000 0110 = **0x2806**

0x2405 => 0010 0100 0000 0101 => Page 4 => Physical Frame 7 (111b)

Physical Address = 0011 1100 0000 0101 => **0x3C05**

- (ii) Which cache block should the CPU check in the cache for the start of the main program? What is the corresponding TAG value used to check for cache hit/miss?

Physical Address used for tagging. Hence, TAG:BLK:OFFSET = 6:5:5

Start address = 0x2806 =

0010	1000	0000	0110
------	------	------	------

TAG value = 001010 = 0xA, BLK = 000000 = 0